Queen's University Belfast
School of Electronics, Electrical Engineering and Computer Science

Software Development Report

Detecting Self-Admitted Security Debt in Software Projects using Natural Language Processing

Blake Harris
Email: bharris06@qub.ac.uk
Student Number: 40323251
Supervisor: Dr. Desmond Greer

April 25, 2025

Contents

List of Figures

Acknowledgements

I would like to express my thanks to my project supervisor, Dr. Desmond Greer, for his invaluable support, guidance, and insightful and thought-provoking feedback throughout the duration of this project. Dr. Greer's expertise and vision were essential in overcoming the challenges faced and questions raised during the research and development process. His perspective served as a fresh analytic lens that not only addressed immediate issues, but also opened up new lines of inquiry, encouraging deeper exploration of potential solutions.

1 Introduction

1.1 Background and Motivation

In software development, the metaphor of Technical Debt is used to describe trade-offs between short-term productivity and long-term code quality. Security Debt, a subset of Technical Debt, refers to deferred security-related tasks to accelerate development timelines, thus potentially leaving systems vulnerable. Security Debt encapsulates both known and unknown security issues that are unaddressed, compromising software integrity.

Developers sometimes acknowledge their trade-offs explicitly in code comments, commit messages, and issue tracker entries; known as Self-Admitted Technical Debt (SATD). In the realm of security, this is referred to as Self-Admitted Security Debt (SASD). Despite the criticality of security, existing tools focus primarily on detecting general Technical Debt, and fail to address security-related concerns specifically, thus creating a gap in automated tools that can identify, analyze, and categorise SASD.

1.2 Problem Statement

Detecting SASD is challenging due to the unstructured nature of textual data found in code comments, commit messages, and issue trackers. Although tools such as SonarQube and Jira address general code quality issues, they lack the mechanisms to target and classify SASD. Furthermore, no existing solution maps SASD findings to established vulnerability frameworks, such as Common Weakness Enumerations (CWEs), which are widely recognised in the domain of software security. This gap poses risks, as unidentified SASD may expose systems to vulnerabilities and exploits.

1.3 Project Objectives

This software development project aims to address these challenges by creating an integrated tool that automates the detection and categorisation of SASD in software repositories. This tool leverages the following:

1. **Natural Language Processing (NLP)** to analyze textual data from comments, commits, and issue entries.
2. **GitHub API** to collect data from repositories for analysis.
3. Mapping mechanisms to associate SASD findings with relevant CWEs, providing actionable insights for developers.

This tool will enable developers to identify vulnerabilities proactively, improve security resilience, and improve code quality during development, code reviews, and maintenance.

1.4 Significance of the Software

The tool developed fills a critical gap in the intersection of Technical Debt management and software security. Unlike existing solutions that overlook the contextual nuances of security debt, this tool focuses specifically on SASD, providing developers with actionable insights into potential vulnerabilities. This proactive approach to security debt management ensures that vulnerabilities can be detected and addressed quickly, thus leading to more robust and secure software systems.

1.5 Scope of the Software

The software comprises three core components:

1. **Backend**: A Flask-based RESTful API responsible for data collection, preprocessing, NLP analysis, and CWE mapping.
2. **Frontend**: A React-based web interface allowing users to input repositories, view results, and interact with SASD analysis visually.
3. **VSCode Extension**: A developer-centric integration enabling in-editor SASD detection and analysis for seamless workflow integration.

These components work cohesively to create a scalable and efficient solution for detecting SASD across various repositories and project structures.

2 Purpose, Operation, and Validation

2.1 Purpose

The main objective of this software tool is to detect and categorize Self-Admitted Security Debt (SASD) within software repositories. By automating SASD identification, the tool addresses the gap in existing solutions that do not specifically focus on security-related technical debt. The key ideas are:

1. To enable developers to identify and understand explicit security-related issues based on code comments, commit messages, and issue trackers.
2. Map detected SASD to Common Weakness Enumerations (CWEs) and provide actionable insights into potential vulnerabilities.
3. Facilitate proactive security management and improve software resilience through early identification and prioritization of vulnerabilities.

The tool is designed to integrate seamlessly into modern software development workflows, offering analysis capabilities via a web interface and a VSCode extension.

2.2 Operation

The software tool operates mainly through two user-oriented interfaces: a web-based frontend and a VSCode Extension.

Through the **web-based frontend**, users provide a repository URL, enabling the software to fetch relevant repository data for analysis. Users can easily navigate repository content, select specific files, and initiate analysis. The analysis process utilises Natural Language Processing to detect and classify instances of Self-Admitted Security Debt. Upon completion, results — including clear descriptions of security concerns and their associations to industry-standard CWE identifiers — are displayed to the user via an intuitive, graphical interface.

Through the **VSCode extension**, users can directly incorporate SASD detection into their everyday development workflow within their IDE. The extension allows for analysis of selected code snippets, entire source code files, and repository-level information (e.g. commit messages and issues), providing feedback on potential security vulnerabilities without disrupting the development process.

The software is essentially operated by users through straightforward, intuitive interactions, requiring minimal technical setup or expertise, thus making SASD detection an integrated and accessible part of the development lifecycle.

2.3 Validation

Validation of the software ensures that it operates reliably, effectively, and as intended. To confirm that the software meets its intended goals, validation procedures focus on two main aspects:

1. **Functional Validation**:
   This ensures that each aspect of the software responds correctly and reliably to user interactions. This includes verifying the retrieval and analysis of repository data, responsiveness of user interfaces, and the accurate display of analysis results.

2. **Accuracy and Reliability**:
   Confirming that the underlying NLP analysis and SASD detection are performed accurately, and provide meaningful and consistent results. This involves testing with representative data from real-world repositories to ensure correct identification and classification of SASD.

3 Design and Implementation

3.1 Assumptions

3.1.1 Functional Assumptions

- **Integration into Developer Workflows**:
  It is assumed that developers will integrate the tool into their existing workflows, similar to tools such as Postman, Docker, database management systems, and code linters. Users are expected to adopt the tool as a standard part of their development process without significant changes to their current methodologies.

- **Multi-Modal Analysis Capabilities**:
  The software tool is designed to perform analyses on different GitHub artifacts. It is assumed that users will be able to select specific artifacts to analyze, and that the feedback from the tool will be sufficiently detailed to support iterative improvements.

- **User Interaction Expectations**:
  It is assumed that users will actively utilize the tool's features to refine their code quality and address security debt as it become apparent. The tool is expected to provide detailed responses including information on how the code contains security shortfalls, mappings to Common Weakness Enumerations (CWEs) and actionable insights to help the developer understand and tackle security debt in their codebase.

3.1.2 Technical and Environmental Assumptions

- **Deployment Platforms**:
  The tool is assumed to be developed as a dual-platform solution, being both an integrated development environment (IDE) extension for Visual Studio Code, and a standalone web application. This design decision presumes that developers have access to both modern code editors and a stable web environment.

- **Connectivity Requirements**:
  A stable internet connection is assumed, and a prerequisite, for the tool.

- **Ephemeral Data Handling**:
  It is assumed that temporary, session-based storage of analysis results is adequate. This design choice is based on the expectation that developers will modify the underlying repository following analysis, thus rendering previous results obsolete.

3.1.3 Performance and Scalability Assumptions

- **Response Times and Resource Utilisation**:
  It is assumed that analysis tasks can be performed within acceptable response times, even under varying repository sizes. The use of session storage is assumed to enhance performance by minimizing redundant API calls for repeated analyses of the same artifacts in a single session.

- **Variable Workloads**:
  Although persistent data storage is not implemented, it is assumed that this is a sufficient design choice as core functionality focuses on providing up-to-date analysis rather than tracking historical security debt trends. In this context, the scalability assumption is that handling transient data per session adequately supports the tool's performance under normal operational loads.

3.1.4 Data and Security Assumptions

- **Model Reliability and Data Consistency**:
  It is assumed that the Natural Language Processing (NLP) model used will consistently and accurately detect instances of Self-Admitted Security Debt, and map them to the most relevant CWE. This is critical, as actionable insights provided are based on the assumption of accurate and consistent model performance.

3.1.5 Operational and Maintenance Assumptions

- **User-Driven Data Refreshing**:
  It is assumed that users will actively modify their code to address highlighted security debt. Therefore, subsequent analyses are expected to produce different results, thus emphasizing the tool's role in iterative code quality improvement, rather than as a static reporting tool.

- **Simplified Deployment**:
  The lack of persistent data storage is assumed to simplify deployment, reducing overheads in backend communication and data management.

- **Simplified Maintenance**:
  The decision to use session storage rather than persistent storage is assumed to reduce maintenance complexity. This approach minimizes backend load and potential data consistency issues over time, thus streamlining both operational support and troubleshooting issues.

3.2 Technologies Used

The developed software uses a combination of technologies and frameworks, selected based on familiarity and comfortability, and ability to ensure robust functionality, seamless integration, and scalability.

3.2.1 Backend Technologies

- **Flask**: A lightweight Python web framework, used to create a RESTful API for data collection, preprocessing, NLP integration, and CWE mapping.
- **Flask-CORS**: Middleware to ensure secure interactions between the backend and React frontend.
- **Pandas**: Utilised for handling and processing structured data during analysis.
- **Requests**: For making HTTP requests to external APIs, e.g. the GitHub API.
- **Python-dotenv**: For securely managing environment variables, e.g. API keys.
- **Pydantic**: Provides robust data validation for API payloads, ensuring consistency and reliability.
- **OpenAI API**: Integrated for Natural Language Processing (NLP) to detect and categorise SASD.

3.2.2 Frontend Technologies

- **React**: A JavaScript library for building dynamic and interaction user interfaces, used to create the web interface.
- **React Router**: Handles client-side routing for navigating between pages.
- **React Icons**: Adds lightweight and scalable icons for the UI.
- **Axios**: A promise-based HTTP client for communicating with the backend API.
- **dotenv**: Used for managing environment variables in the frontend.

3.2.3 VSCode Extension Technologies

- **TypeScript**: A strongly-typed superset of JavaScript that provides type safety and maintainability for the extension.
- **VSCode API**: Used to create commands and interactions with the VSCode editor to enable the core features for SASD detection.
- **Axios**: Facilitates communications with the backend API.
- **dotenv**: Manages environment variables required for configuration.

3.2.4 Containerisation and Orchestration

- **Docker**: Used to containerise the backend and frontend services to ensure consistent deployment across both development and production environments.
- **Docker Compose**: Manages multi-container setups to allow the backend and frontend to run seamlessly together within a shared network.

3.2.5 Additional Tools

- **GitHub API**: Provides repository data, e.g. commit messages, issue tracker entries, and raw code files for analysis.
- **Common Weakness Enumerations (CWEs)**: Used as a framework to categorise and describe identifed security vulnerabilities.

3.3 System Design

This section highlights the system's architecture to show how the tool integrates its components and external services to efficiently detect Self-Admitted Security Debt.

3.3.1 High Level Architecture

The system architecture is designed to efficiently integrate multiple distinct components through well-defined interfaces, ensuring seamless functionality in identifying and categorising Self-Admitted Security Debt (SASD). The architectural framework is built around containerised services managed by Docker, ensuring consistent deployment and scalability across diverse environments.

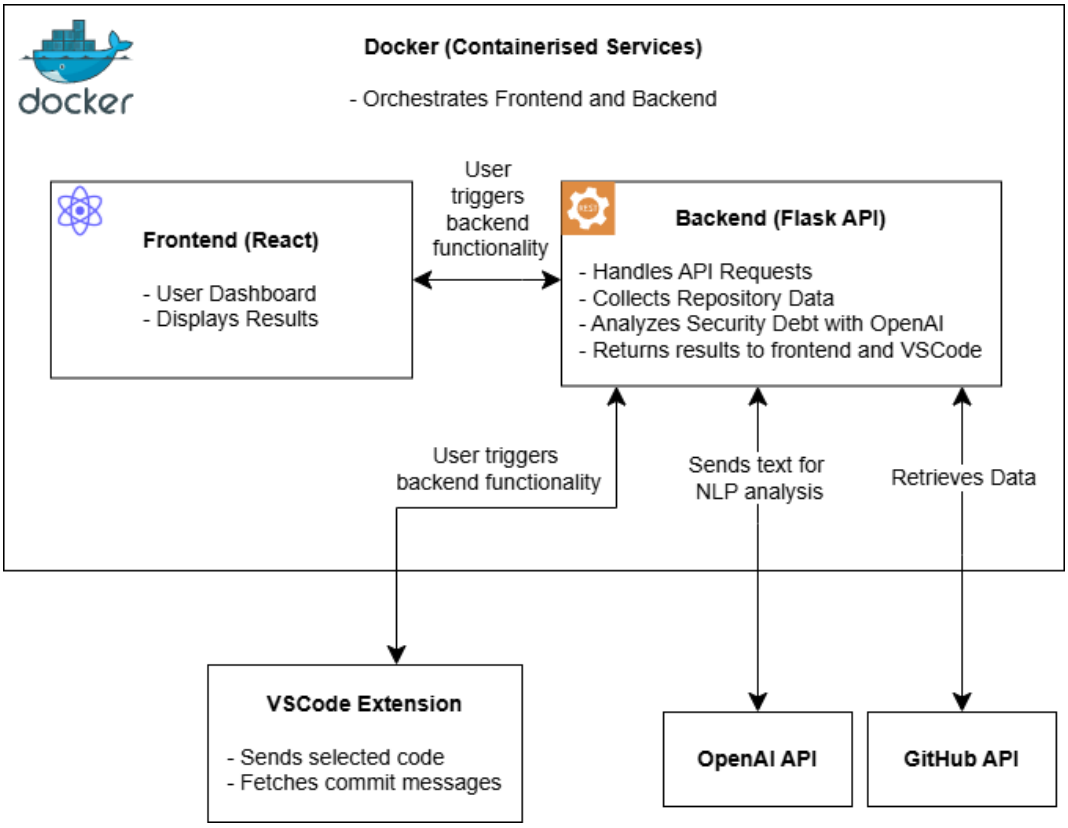


Figure 1: High Level Architecture Diagram

From a high level, the main components and their interactions are as follows:

1. **Docker Containerisation**:
   Docker orchestrates the deployment and management of the frontend and backend ser-

vices. Containerisation allows isolated environments for each service, providing portability, consistency, and efficient resource allocation and utilisation.

2. **React Frontend**:
   The frontend is built using *React* and provides the user-facing interface of the software. It consists of two core functionalities:

   - **User Dashboard**: Enables users to initiate analyses, manage project data, and oversee detection processes.
   - **Result Display**: Presents the outcomes of SASD detection and categorisation, including mappings to Common Weakness Enumerations (CWEs), allowing users to review vulnerabilities and plan effectively.

3. **Flask Backend**:
   The backend service, built using the Flask framework, handles all business logic. Key functions include:

   - **API Request Handling**: Receives and processes requests from the frontend and VSCode Extension.
   - **Repository Data Collection**: Interfaces with the GitHub API to retrieve commit messages, issue entries, and code comments for analysis.
   - **Security Debt Analysis**: Utilises the OpenAI API for Natural Language Processing (NLP) to detect and analyse SASD instances within collected data. Then pipelines results back into the API for CWE categorisation.
   - **Result Generation and Distribution**: Aggregates and structures analysis outcomes, and returns comprehensive results to both the frontend and VSCode extension.

4. **VSCode Extension**:
   A developer-centric integration within the Visual Studio Code IDE. This component enhances developer workflows by:

   - Allowing developers to trigger backend functionality directly from the IDE.
   - Sending selected code snippets to the backend service for immediate analysis.
   - Presenting SASD detection results and CWE categorisations directly within the development environment, allowing for proactive security management.

3.3.2 Component Diagram

The below UML component diagram depicts how the system's components interact with each other through defined interfaces, focusing on their dependencies and the services that they provide. "Interfaces", in this context, represent "contracts" that define functionality that a component provides to and requires from another component.

- **Frontend Components**
  - **BackendActions**: Facilitates calls to the backend, providing the *ITriggerAnalysis* interface to initiate analysis requests via API interactions (*IMakeAPICalls*).
  - **API Service**: Manages HTTP requests through *IMakeAPICalls*, exposing the backend API to frontend components via the *IExposeAPI* interface.

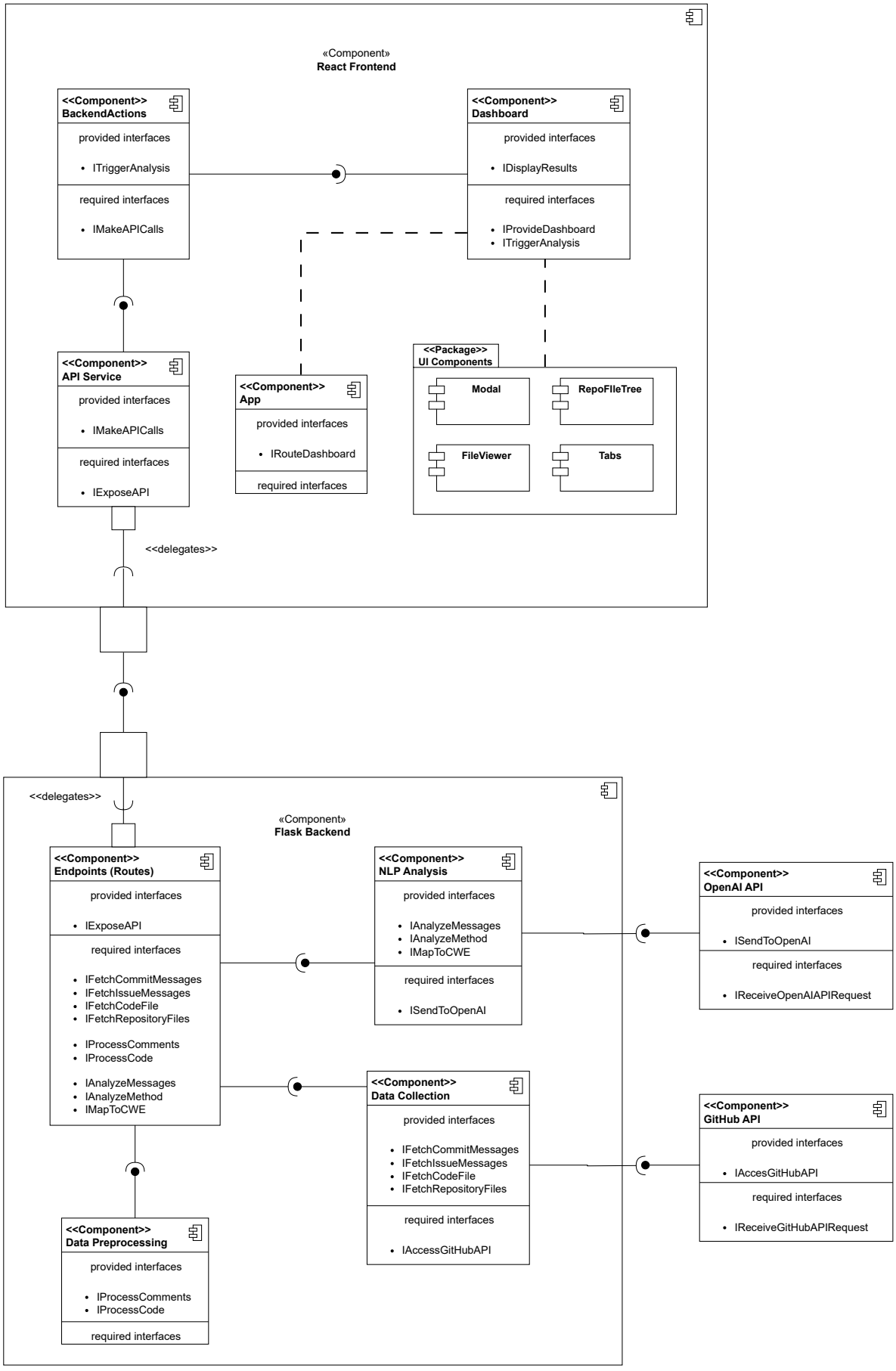


Figure 2: Component Diagram

  - **Dashboard**: Provides the main user interface through the *IDisplayResults* interface, relying on backend services for data (*IProvideDashboard*) and triggering analysis (*ITriggerAnalysis*).
  - **UI Components Package**: Contains modular UI components such as *Modal*, *FileViewer*, *RepoFileTree*, and *Tabs*, enhancing the visual representation and interactivity of analysis results.
  - **App Component**: The main entry point of the application, providing the routing interface (*IRouteDashboard*) to manage user navigation and interactions.

- **Backend Components**
  - **Endpoints (Routes)**: Exposes the API (*IExposeAPI*) to the frontend, delegating specific tasks such as fetching data, processing comments, analyzing data, and mapping to CWEs.
  - **Data Collection**: Provides data fetching operations, interacting with GitHub (*IFetchCommitMessages*, *IFetchIssueMessages*, etc) through *IAccessGitHubAPI*.
  - **Data Preprocessing**: Prepares data for NLP analysis, handling the processing of code comments (*IProcessComments*).
  - **NLP Analysis**: Interfaces with OpenAI to analyze security debt (*IAnalyzeMessages*, *IAnalyzeMethod*) and to categorize identified debt (*IMapToCWE*) via the *ISendToOpenAI* interface.

- **External Components**
  - **OpenAI API**: Receives and processes NLP analysis requests, returning structured responses through interfaces like *ISendToOpenAI* and *IReceiveOpenAIAPIRequest*.
  - **GitHub API**: Provides repository data via interfaces such as *IAccessGitHubAPI* and *IReceiveGitHubAPIRequest*, required for SASD analysis.

3.3.3 Deployment Diagram

The deployment diagram outlines the physical deployment strategy and runtime environment:

- **Developer Machine**: Developers run the VSCode Extension locally or use the web tool, integrating seamlessly with the backend service via network interactions.
- **Production Cloud Server**: The React frontend and Flask backend are deployed as containerised artifacts managed by Docker Compose, communicating internally via Docker networking.
- **External Services**: The backend integrates externally hosted APIs — OpenAI Cloud (for NLP analysis) and GitHub cloud (for repository data retrieval) — establishing network-based interactions with cloud services that are essential for the functionality and scalability of the system.

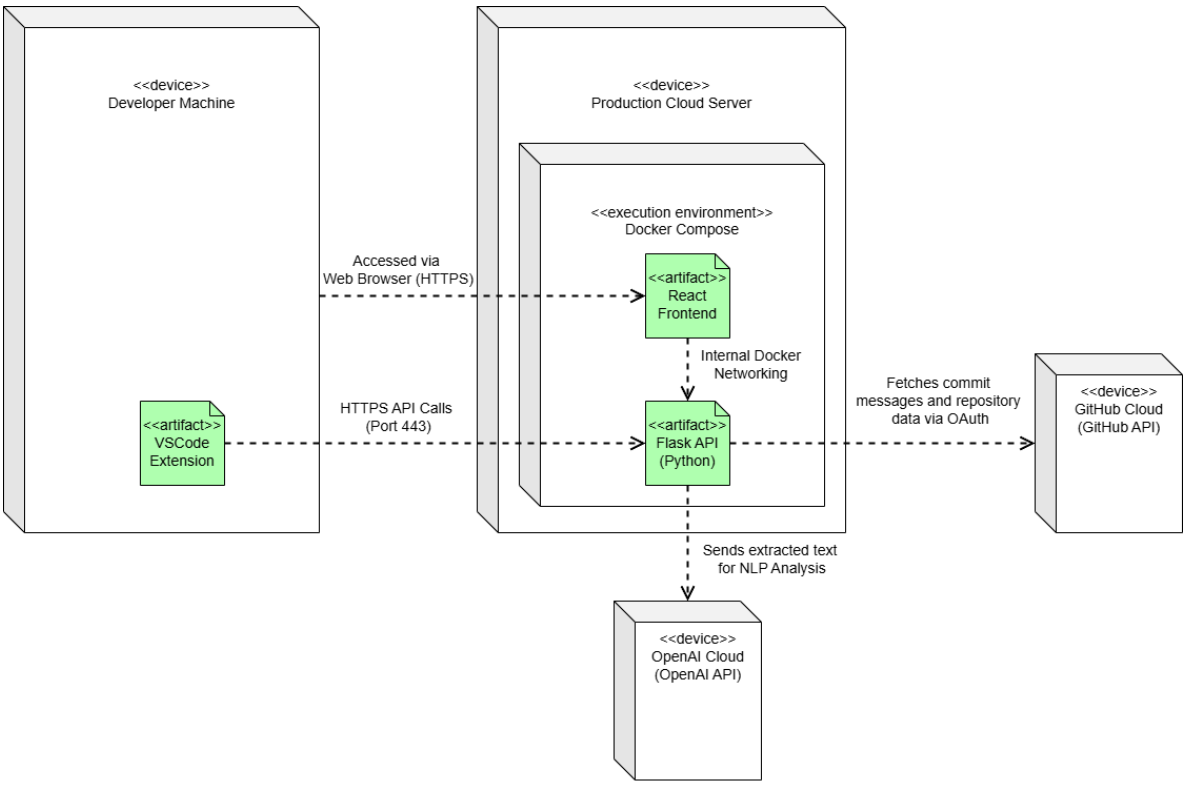


Figure 3: Deployment Diagram

3.3.4 Use Case Diagram

The UML Use Case diagram depicts how end users (or external systems) interact with the Flask backend to perform tasks related to analysing and managing Self-Admitted Security Debt (SASD). Each use case corresponds to a distinct workflow that is initiated by a user through the frontend or VSCode extension. The diagram highlights the central role played by the Flask backend in orchestrating repository data retrieval, code analysis, and reporting.

1. **Actors**

   - **Developer/User**:
     Interacts with the system through the React frontend or VSCode extension. The user can trigger the different analysis tasks, view results, and navigate through the repository artifacts.

2. **Use Cases**

   - **Fetch Repository Structure**: Allows the user to retrieve the directories and files from a GitHub repository.
   - **View Code file**: Retrieves the file content from a user-selected file from the repository and displays it to the frontend.
   - **Analyze Selected Code**: Invokes the backend to examine a user-highlighted code segment's comments.
   - **Analyze Entire File**: Invokes a broad scan of an entire code file's comments.

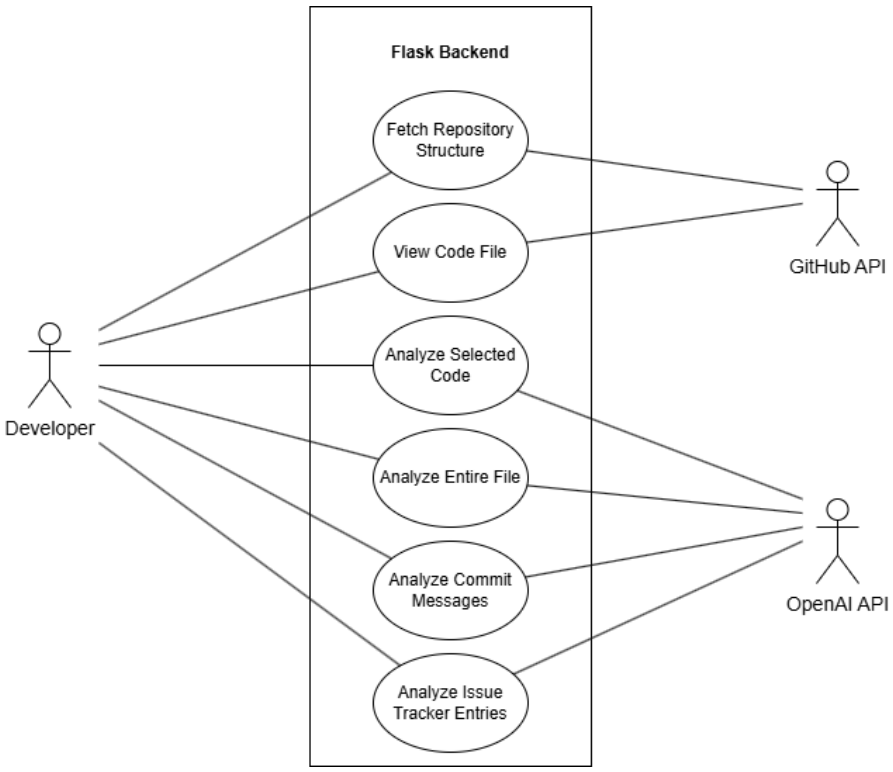


Figure 4: Use Case Diagram

- **Analyze Commit Messages**: Fetches and performs analysis of historical commit messages from the repository.
- **Analyze Issue Tracker Entries**: Fetches anf performs analysis of issue entries from the repository.

3. **Relationships and Flow**

- **Extends/Includes**: In a typical use case model, certain actions (e.g. retrieving repository data) may be a prerequisite for other actions (e.g. analyzing a code file). Although not specifically depicted in *Figure 4*, these conceptual relationships exist logically:
  - *Fetch Repository Structure* precedes *View Code File*, which in turn may lead to *Analyze Selected Code*.
- **Interactions**: The user enters a public repository URL which fetches the repository structure, then selects a file. The desired operation dispatches a request to the Flask backend. The backend communicates with GitHub (for data retrieval) and OpenAI (for NLP analysis), before returning actionable insights to the user.

3.4 Data Models

3.4.1 Pydantic Schemas for API Communication

The employed data modeling strategy is based on Python's Pydantic library [1]. Several classes are defined that inherit from Pydantic's *BaseModel* to represent the expected structure of API requests and responses. These models include:

- **Request Models**
  Models, such as *AnalyzeCommitsRequest*, *AnalyseIssuesRequest*, *AnalyzeMethodRequest*, and *AnalyzeCodeFileRequest*, encapsulate the required inputs for the various endpoints. Each field in these models is annotated with type hints and supplemented with metadata — via the *Field* functions — which provides example values and detailed descriptions. This enforces type safety and data integrity at runtime, and also supports automatic generation of API documentation.

- **Response Models**
  Response models, such as *CWEAnalysisResponse*, *MethodAnalysisResponse*, and *CodeFileAnalysisResponse*, define the structure of the outputs returned by the API. By ensuring that all responses adhere to a pre-defined format, these models act as a contract between the API and its consumers, ensuring consistent communication and easy integration with other systems.

The use of Pydantic schemas is exemplified in the backend implementation, where each API endpoint instantiates the appropriate Pydantic model to parse and validate incoming JSON payloads. This design choice enforces a strict contract for data integrity and decouples data validation from core business logic, thus promoting modularity and maintainability. In practice, any deviation from the expected data format is immediately captured as a validation error, ensuring that only correctly structured data proceeds through the application.

This approach bolsters reliability through rigorous type-checking and also streamlines error handling, as Pydantic's built-in mechanisms return consistent, detailed error messages. The inclusion of metadata — such as example data and descriptive annotations — allows for automatic documentation generation, significantly improving the developer experience.

Overall, integrating Pydantic schemas creates a robust and scalable framework for managing API data, and ensures that updates to data structures are handled in one central location, thus reducing maintenance overheads and supporting future system development.

3.5 Code Details

3.5.1 Modular Architecture and Initialization

The system architecture involves a clear **separation of concerns**, embodying the **Single Responsibility Principle** from the SOLID principles. For example, the **create_app** function in *main.py* is responsible for initializing the Flask application, applying configuration settings (via the **Config** class in *config.py*), enabling CORS, and registering the API blueprint from *endpoints.py*. This modular approach helps ensure that configuration and routing are **decoupled** from the core business logic, allowing for unit testing and maintenance. By abstracting application configuration into the Config class, the **Open/Closed Principle (OCP)** is utilised, as the system can have additional configurations with modifying the existing initialization logic. This modular structure **reduces coupling** across system components and **improves overall cohesion** within modules.

The initialization and configuration approach is also an implementation of the **Factory Pattern**, as the create_app function encapsulates the creation and initialization logic of the Flask app, and returns a fully configured instance.

3.5.2 Data Collection and External Communication

(a) **API Integration and Recursion**
Several functions are dedicated to interfacing with the GitHub API to collect the required data. Notably:

- **fetch_commit_messages**: retrieves commit messages.
- **fetch_issue_messages**: retrieves issue details.
- **fetch_raw_code**: decodes Base64-encoded content from code files.
- **fetch_raw_files**: implements a **recursive algorithm** to traverse nested repository directories, ensuring that the entire repository structure is processed regardless of depth.

The **KISS** principle is used here to ensure each function has a clear, focused purpose, minimizing complexity and enhancing readability and maintainability. The clear separation and minimal inter-dependencies between these functions show **strong cohesion and minimal coupling**.

3.5.3 Code Parsing and Preprocessing

(a) **Regex-Based Method Extraction**
A core functionality is parsing raw source code to extract definitions and their bodies. The function **process_code** in *data_preprocessing.py* uses a sophisticated regular expression to identify method signatures — including access modifiers, return types, and parameter lists — and maintains state to track the beginning and end of a method block. This ensures that multi-line methods are accurately captured even when the formatting is varied.

Key Considerations:

- **Stateful Parsing**: The parsers keeps track of when it is inside a method block to ensure all relevant code lines, and most importantly comment lines, are correctly aggregated, aligning with principles of **algorithmic clarity**.
- **Regex Complexity**: The regex is designed to accommodate diverse method signatures.
- **Extensibility**: Despite being optimised for Java syntax, the design can be easily adapted for other programming languages with minor changes to the regex and state logic, adhering again to the **Open/Closed Principle**.

(b) **Comment Extraction**
Following method extraction, the function **process_comments** (also in *data_preprocessing.py*) filters code lines based on a specified comment identifier. By isolating inline annotations, the function allows a focused analysis on those comments where developers may have implicitly, or explicitly, acknowledged security shortfalls — i.e. self-admitted security debt. The specificity and simplicity here again reflects the **KISS principle** being employed, ensuring targeted and straightforward processing of data for NLP tasks.

3.5.4 NLP Integration for Security Analysis

(a) **Detecting Self-Admitted Security Debt (SASD)**

- **analyze_messages** and **analyze_method** in *nlp.py* are the methods designed to process textual data from commit messages, issue messages, and code comments.
- These functions prompt the NLP model to evaluate whether the text indicates security debt (either implicitly or explicitly). The responses are engineered to be binary ("Yes" or "No") with concise justifications, thus directly addressing the main research objective to detecting SASD.

(b) **Categorisation Using CWEs**

- If SASD is detected, **map_to_cwe** (also in *nlp.py*) is responsible for taking the analysis output and pipelining it back into the NLP model. The model is prompted to map the original explanation of the detected security debt to the most relevant classification in the CWE framework.
- This process not only aligns the security debt with industry-recognized categories, but also provides a structured method for categorizing the identified issues, thus addressing another research goal.

(c) **Other Features**:

- **Prompt Engineering**: Prompts are carefully created to ensure that the NLP model concentrates on explicit security-related language, critical for accurate SASD detection.
- **API Call Resilience**: Interactions with the NLP service are wrapped in *try/catch* blocks to gracefully handle exceptions — ensuring that network issues or API timeouts don't disrupt the analysis workflow.

3.5.5 Error Handling and Data Validation

Error handling is extensive throughout the backend, implemented at several layers:

- **Input Validation**: As discussed previously, Pydantic models ensure that incoming data adheres to strict formats. This validation prevents malformed data from progressing through the system by using the **Fail-Fast Principle** to quickly identify and reject invalid inputs before processing begins.
- **Targeted Exception Handling**: In the API endpoints, *try/catch* blocks catch both Pydantic **ValidationError** and generic **Exception** types. This layered error management strategy isolates the point of failure, and also return the appropriate HTTP status codes (400 for validation issues, 500 for general exceptions).
- **External API Error Checks**: Functions that interact with external services (e.g. GitHub or OpenAI APIs) verify response statuses before proceeding to ensure that issues such as API rate limits or malformed responses are caught early and are unable to propagate through the system - an example of **Defensive programming**.

3.5.6 Engineering Implications

- **Separation of Concerns**:
  By isolating configuration, data retrieval, data parsing, and NLP analysis, the system is highly modular, maintainable, and scalable, achieving low coupling, high internal cohesion, and aligning well with SRP.
- **Algorithmic Robustness**:
  The recursive algorithm for file retrieval and the regex-based approach for method extraction address the complexity involved in parsing variable code formats and nested repository structures, and highlight strong algorithmic practices.
- **Resilience and Scalability**:
  Comprehensive error handling across multiple layers — input validation, targeted exception handling, and external API checks — ensure that the system remains robust and reliable even under strenuous conditions.

- **Software Design Principles**:
  Use of principles such as **Open/Closed Principle (OCP)**, **Single Responsibility Principle (SRP)**, **KISS**, and **Defensive Programming** throughout development ensures high-quality, maintainable, and extensible software.

3.6 User Interface

The user interface (UI) for this project is designed as a graphical user interface (GUI) developed using the React framework. It emphasizes usability, clarity, and responsiveness, providing users with a straightforward, intuitive, yet powerful environment to interact with the underlying backend services.

3.6.1 Overall Design and Layout

The UI is structured for intuitive navigation and efficient interaction. It clearly presents defined areas of functionality, including repository input, file exploration, and result presentation. Each piece of functionality is encapsulated in dedicated React components, aligning well with the **Single Responsibility Principle (SRP)**, creating a modular and maintainable frontend architecture.

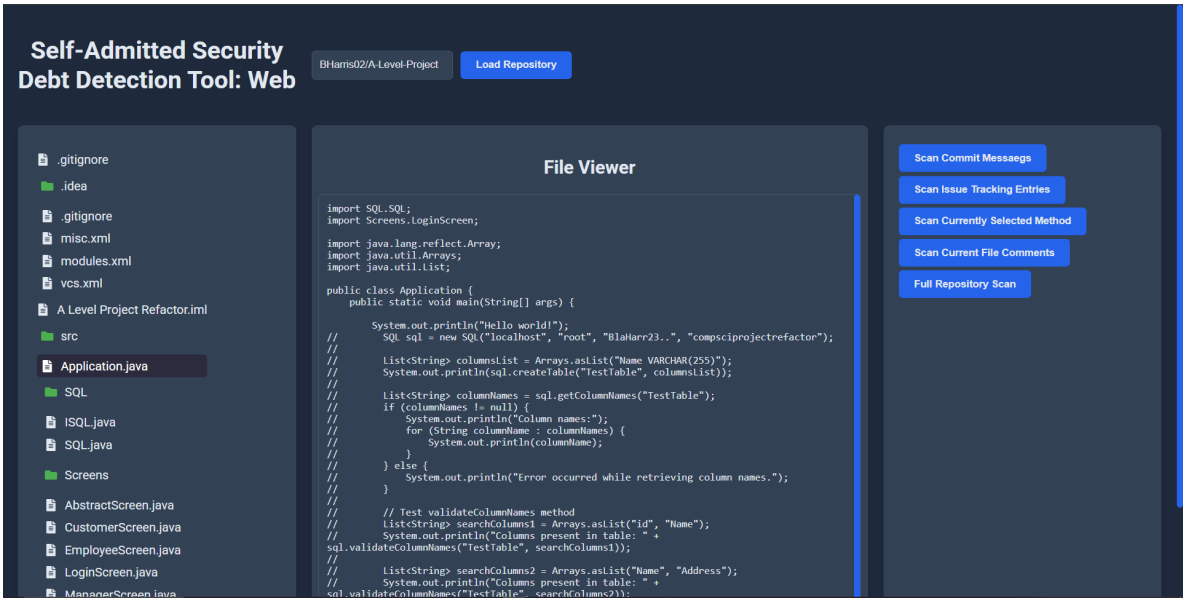


Figure 5: User Interface Layout

3.6.2 Primary UI Components

Each visual element of the frontend are organised into separate React components, each serving a unique purpose:

1. **RepoInput**: This component provides a straightforward input form for GitHub repository URLs. Its design simplicity ensures clarity and reduces error - i.e. the **KISS** principle.

2. **RepoFileTree**: Implements a hierarchical file explorer to allow users to navigate repository structures. This component uses **recursive** rendering logic for directory traversal, enhancing usability, and having strong algorithmic robustness.

3. **Tabs**: This component organizes analytical output into distinct, labeled sections, providing an efficient way to manage multiple sets of results.

4. **CommitAnalysis, IssueAnalysis, and CodeAnalysis**: Each of these components is dedicated to displaying specific analysis results in a structured way. Each component has **high cohesion**, as they explicitly focus on their respective analysis domains.

5. **FileViewer**: This provides users with detailed views of file content directly in the interface, thus enhancing the interactive experience by clearly separating raw and analyzed data.

6. **Modal**: Used for displaying supplementary information or alerts to enhance usability without disrupting the user's workflow. This demonstrates effective use of UI overlays to help maintain context and reduce cognitive load.

3.6.3 Software Design Principles and Patterns

- **Component-Based Design**: The UI is built from discrete, reusable components, utilizing React's core architectural philosophy and improving maintainability, testability, and scalability.

- **Single Responsibility Principle (SRP)**: Each React component has a clearly defined role in the UI, reducing complexity and allowing for easy future modifications.

- **State Management**: State handling in the components, especially the **Tabs** and **Modal** components, is carefully managed to ensure predictable and responsive UI behaviour.

- **High Cohesion and Loose Coupling**: The components are designed to be internally cohesive, focusing on their specific tasks, with minimal dependency on external components, thus improving system modularity.

- **Rendering and Recursion**: Components like **RepoFileTree** use recursion and conditional rendering to manage complex data structures and dynamically changing UI elements, showing solid algorithmic practices.

3.6.4 Integration with Existing VSCode UI

It should be noted that the system's VSCode extension relies on the pre-existing Visual Studio Code user interface for interactions. The project does not require the design or implementation of additional UI components specifically for the extension. This respects existing standards and user familiarity with VSCode.

3.7 Quality Assurance

3.7.1 Version Control and Versioning

All project components — including the backend (Flask API), frontend (React), VSCode extension, and orchestration components — are version controlled using **GitLab**, each each having its own dedicated repository, a decision made to improve project management and

modularity. This separation ensures that if a bug occurs, or changes are required, in one part of the software, the other components will be unaffected, thus allowing for better troubleshooting and updates. Using Git in this way provides better change-tracking, allows for easy rollbacks, and helps maintain organised development workflows.

3.7.2 Automated Documentation Generation

Automated documentation generated was implemented using **Flask-Pydantic-Spec** integrated with **Swagger UI**. Each API endpoint uses Flask-Pydantic decorators referencing Pydantic schemas for requests and responses to automatically produce as OpenAPI specification. Swagger UI provides an interactive, web-based interface that allows for real-time testing and validation of the API's endpoints directly from the browser. Any changes made to the endpoint schemas or logic will automatically update the documentation, thus ensuring consistency between code and documentation.

3.7.3 Testing

Testing was a core part of the assuring quality software. The testing strategy incorporated **unit testing**, **integration testing**, and **code coverage analysis**. Python's built-in **unittest** framework was used to implement test cases, alonnside **unittest.mock** for isolating dependencies and **coverage.py** for measuring test coverage.

- **Unit Testing**
  Unit tests were designed to isolate the ensure individual functions operate correctly. This included:
  - **Data Collection Module**:
    Functions such as **fetch_commit_messages**, **fetch_issue_messages**, and **fetch_raw_code** were tested to ensure correct data parsing and interactions with the GitHub API. External HTTP requests were mocked to simulate GitHub responses.
  - **Data Preprocessing Module**:
    Test cases validated accurate extraction of individual methods, with their comments, from raw code strings. These tests used representative code samples to ensure returned data structured matches the intended output, including method signatures and bodies.
  - **NLP Module**:
    NLP module tests targeted LLM interactions, mocking OpenAI's API responses to simulate SASD detection and CWE classification. The tests ensured that SASD flags and explanations were correctly parsed and structured. Additional checks ensure that downstream functions, i.e. mapping to CWE identifiers, behaved predictably when nested inside other functions.

  Mocks were used to isolate logic under test and eliminate external service dependencies, creating a deterministic and performant test suite. The use of **assertions** ensured content accuracy and Pydantic schemas compliance, and tests included different inputs to capture both normal and edge cases.

- **Integration Testing**
  Integration tests were implemented using Flask's test client to simulate HTTP requests and validation endpoint behaviour under realistic scenarios. Each test case targeted a specific endpoint route, and all routes were tested. External dependencies, such as

GitHub API calls, code analysis functions, and OpenAI responses, were again mocked using **unittest.mock.patch** to ensure independence from network and third-party services.

The tests validated endpoint response structures, status code, and behavior with valid and edge-case inputs. For example, **/analyze/repo** tests confirmed that commit messages, issues, and code files were processed end-to-end with appropriate NLP analysis and CWE mappings. Similarly, file content and repository structure retrieval endpoints were verified to parse repository data correctly and return the expected hierarchical format.

These integration tests provided assurances that the API correctly orchestrates the backend utilities, gracefully handles exceptions, and returns responses that adhere to the formats defined in the documented OpenAPI specification.

```
(.venv) PS C:\Users\Blake\Desktop\CSC4006-SASD-Detection-Tool\CSC4006-SASD-Detection-Tool-Backend> python -m unittest -v
test_fetch_commit_messages (tests.test_data_collection.TestDataCollection.test_fetch_commit_messages) ... ok
test_fetch_issue_messages (tests.test_data_collection.TestDataCollection.test_fetch_issue_messages) ... ok
test_fetch_raw_code (tests.test_data_collection.TestDataCollection.test_fetch_raw_code) ... ok
test_fetch_raw_files (tests.test_data_collection.TestDataCollection.test_fetch_raw_files) ... ok
test_process_code (tests.test_data_preprocessing.TestDataPreprocessing.test_process_code) ... ok
test_process_comments (tests.test_data_preprocessing.TestDataPreprocessing.test_process_comments) ... ok
test_analyze_commits (tests.test_endpoints_integration.TestEndpointsIntegration.test_analyze_commits) ... ok
test_analyze_commits_exception (tests.test_endpoints_integration.TestEndpointsIntegration.test_analyze_commits_exception) ... ok
test_analyze_commits_validation_error (tests.test_endpoints_integration.TestEndpointsIntegration.test_analyze_commits_validation_error) ... ok
test_analyze_file (tests.test_endpoints_integration.TestEndpointsIntegration.test_analyze_file) ... ok
test_analyze_file_exception (tests.test_endpoints_integration.TestEndpointsIntegration.test_analyze_file_exception) ... ok
test_analyze_file_validation_error (tests.test_endpoints_integration.TestEndpointsIntegration.test_analyze_file_validation_error) ... ok
test_analyze_issues (tests.test_endpoints_integration.TestEndpointsIntegration.test_analyze_issues) ... ok
test_analyze_issues_exception (tests.test_endpoints_integration.TestEndpointsIntegration.test_analyze_issues_exception) ... ok
test_analyze_issues_validation_error (tests.test_endpoints_integration.TestEndpointsIntegration.test_analyze_issues_validation_error) ... ok
test_analyze_method (tests.test_endpoints_integration.TestEndpointsIntegration.test_analyze_method) ... ok
test_analyze_method_exception (tests.test_endpoints_integration.TestEndpointsIntegration.test_analyze_method_exception) ... ok
test_analyze_method_validation_error (tests.test_endpoints_integration.TestEndpointsIntegration.test_analyze_method_validation_error) ... ok
test_analyze_repo (tests.test_endpoints_integration.TestEndpointsIntegration.test_analyze_repo) ... ok
test_analyze_repo_exception (tests.test_endpoints_integration.TestEndpointsIntegration.test_analyze_repo_exception) ... ok
test_analyze_repo_missing_repo (tests.test_endpoints_integration.TestEndpointsIntegration.test_analyze_repo_missing_repo) ... ok
test_file_content (tests.test_endpoints_integration.TestEndpointsIntegration.test_file_content) ... ok
test_repo_structure (tests.test_endpoints_integration.TestEndpointsIntegration.test_repo_structure) ... ok
test_analyze_messages_exception (tests.test_nlp.TestNLPModule.test_analyze_messages_exception) ... ok
test_analyze_messages_mocked (tests.test_nlp.TestNLPModule.test_analyze_messages_mocked) ... ok
test_analyze_messages_with_sasd_detection (tests.test_nlp.TestNLPModule.test_analyze_messages_with_sasd_detection) ... ok
test_analyze_method_exception (tests.test_nlp.TestNLPModule.test_analyze_method_exception) ... ok
test_analyze_method_mocked (tests.test_nlp.TestNLPModule.test_analyze_method_mocked) ... ok
test_map_to_cwe_exception (tests.test_nlp.TestNLPModule.test_map_to_cwe_exception) ... ok
test_map_to_cwe_mocked (tests.test_nlp.TestNLPModule.test_map_to_cwe_mocked) ... ok

----------------------------------------------------------------------
Ran 30 tests in 0.069s

OK
```

Figure 6: Testing output

- **Code Coverage**
  Test coverage reports generated using **coverage.py** provide insights into the extent of test execution across the codebase, highlighting missed statements; areas that require further testing. The final report indicated an overall backend coverage of **94%**. Key modules such as **data_collection.py**, **schemas.py**, and **nlp.py** achieved full coverage. Modules such as **data_preprocessing.py** and **endpoint.py** achieved a high coverage, though a few lines were untested. These were primarily very low-probability conditions, such as exception handling branches or unbalanced code input scenarions. For example:

  – In **endpoints.py**, uncovered lines included exception blocks or branches where the GitHub API returns an invalid response or where request fields are missing, both of which will be caught by Pydantic and return a 422 status code, which is covered elsewhere.

– In **data_preprocessing.py**, a missed condition involved appending an unclosed function at the end of the file — specifically, a function missing a closing bracket. This is a low-probability occurrence as we generally assume a competent developer workflow, supported by modern IDEs that auto-complete or flag unbalanced brackets during development. Therefore, encountering malformed code in practise is rare, and this untested branch represents minimal risk.

```
(.venv) PS C:\Users\Blake\Desktop\CSC4006-SASD-Detection-Tool\CSC4006-SASD-Detection-Tool-Backend> coverage report -m
Name                                Stmts   Miss  Cover   Missing
-----------------------------------------------------------------
api\__init__.py                         0      0   100%
api\routes\__init__.py                  0      0   100%
api\routes\endpoints.py               144     15    90%   42, 59, 82, 110, 124, 168, 184, 217, 231, 243, 274-275, 288, 298-299
api\utils\__init__.py                   0      0   100%
api\utils\config.py                    16      1    94%   21
api\utils\data_collection.py           35      0   100%
api\utils\data_preprocessing.py        32      2    94%   50, 75
api\utils\nlp.py                       38      0   100%
api\utils\schemas.py                   45      0   100%
-----------------------------------------------------------------
TOTAL                                 310     18    94%
```

Figure 7: Code Coverage

3.7.4 Bug Tracking and Resolution

Due to the scale of the project and the small number of bugs identified during development, not formal bug tracking system was utilised. Instead, issues were informally tracked on paper and addressed during development. A few additional bugs were detected during the testing process, which were also promptly dealt with. This approach was most suitable given the project's scale and complexity.

3.7.5 Continuous Integration & Deployment

A `.gitlab-ci.yml` was created to run tests and build Docker images upon pushing changes to the repository, but in absence of available GitLab runners the pipeline never executed. Instead, container builds and test runs were performed locally via Docker Compose.

3.7.6 Code Style & Conventions

No formal linters (e.g. Flake8, ESLint) or auto-formatters (e.g. Black, Prettier) were used due to the solo scope of this project. Source files were formatted using default IDE settings and manual checks.

3.7.7 Code Reviews

Due to the solo nature of the project, no peer-review or merge-request workflow was utilised. All commits were self-reviewed before pushing to the repository.

3.7.8 System, Functional & Acceptance Testing

Due to time pressure, no formal System, Functional or Acceptance test suites (e.g. Selenium, Cypress) were developed, and testing instead focused on Unit and Integration coverage. Manual end-to-end validation was performed for key workflows. Implementing automated E2E and acceptance tests remains as planned future work.

A GitLab Documentation

A.1 Orchestration Repository README

README.md 2025-04-25

SASD Detection Tool - Orchestration Repository

This repository contains the orchestration configuration for the SASD Detection Tool, a comprehensive, developer-centric system designed to automatically detect Self-Admitted Security Debt (SASD) in software projects. The system integrates multiple components - including the backend, frontend, and a VSCode extension - into a unified, containerised environment using Docker Compose.

This orchestration repository serves as the entry-point for deploying the complete system in a production or development environment.

Table of Contents

Project Structure

The tool is divided into the following repositories:

- **Backend (Flask API):** Handles data retrieval from GitHub, preprocesses textual data, and performs NLP-based detection and CWE mapping.
- **Frontend (React):** Provides a web-based user interface where users can provide repository details, initiate analyses, and view results.
- **VSCode Extension:** Integrates SASD detection into Visual Studio Code, allowing developers to analyze selected code, commit messages, issues, etc, directly in their IDE.
- **Orchestration (This Repository):** Contains the Docker Compose configuration and supporting scripts needed to deploy the backend and frontend together as a single system.
- **(Optional) Experimental Utilities:** Contains code related to experiments conducted in relation to the accompanying research article.

Getting Started

Prerequisites

Ensure that your system has the following installed:

- **Git:** For cloning the repositories.
- **Docker & Docker Compose:** For deploying the services in a containerised environment.
- **Visual Studio Code:** For installing and running the VSCode Extension.

Cloning the Repositories

- Make a directory to store the cloned repositories:

```
mkdir sasd-detection-tool
cd sasd-detection-tool
```

- Clone the **four** required repositories:

```
git clone https://gitlab.eeecs.qub.ac.uk/40323251/CSC4006-SASD-Detection-Tool.git
# backend
git clone https://gitlab.eeecs.qub.ac.uk/40323251/csc4006-sasd-detection-tool-
frontend.git              # frontend
git clone https://gitlab.eeecs.qub.ac.uk/40323251/csc4006-sasd-detection-tool-
orchestration.git         # orchestration
git clone https://gitlab.eeecs.qub.ac.uk/40323251/csc4006-sasd-detection-tool-
vscode-extension.git      # vscode extension
```

Configuration

Create a .env file in the root directory of the Orchestration repository. Use the following template:

```
FLASK_ENV=development

#backend
GITHUB_API_URL = "https://api.github.com"
GITHUB_API_TOKEN = <your_github_api_token>

GITHUB_CLIENT_ID = <your_github_client_id>
GITHUB_CLIENT_SECRET = <your_github_client_secret>

GITHUB_ACCESS_TOKEN_URL = "https://github.com/login/oauth/access_token"
GITHUB_API_USER_URL = "https://api.github.com/user"

OPENAI_MODEL = "gpt-3.5-turbo"
OPENAI_KEY = <your_openai_key>

#frontend
```

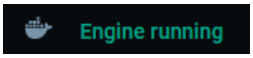
```
NODE_ENV = development
REACT_APP_API_BASE_URL = http://127.0.0.1:5000/api
```

- **GITHUB_API_TOKEN:** A personal access token from GitHub for API calls.
- **GITHUB_CLIENT_ID:** Unique ID for the signed-in user.
- **GITHUB_CLIENT_SECRET:** Used to get an access token for the signed-in user.
- **OPENAI_KEY:** Your OpenAI API key to perform tasks using NLP.

Deploying the System using Docker Compose

The orchestration repository provides a Docker Compose configuration that combines the backend and frontend components into a unified deployment.

- Ensure the Docker engine is running:



- Navigate to the Orchestration repository:

```
cd csc4006-sasd-detection-tool-orchestration
```

- Use Docker Compose to deploy the frontend and backend as a unified system.

```
docker-compose up --build -d
```

- You will be able to access the full system at the following address:

```
http://127.0.0.1:3000
```

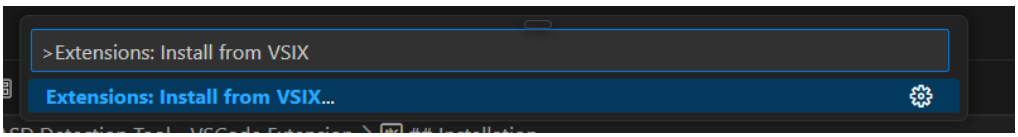
VSCode Extension Integration

Although the VSCode extension is managed in its own repository, it is designed to work with the deployed backend API. Once you have the full system running via Docker Compose, install the VSCode extension to integrate SASD detection into your development workflow.

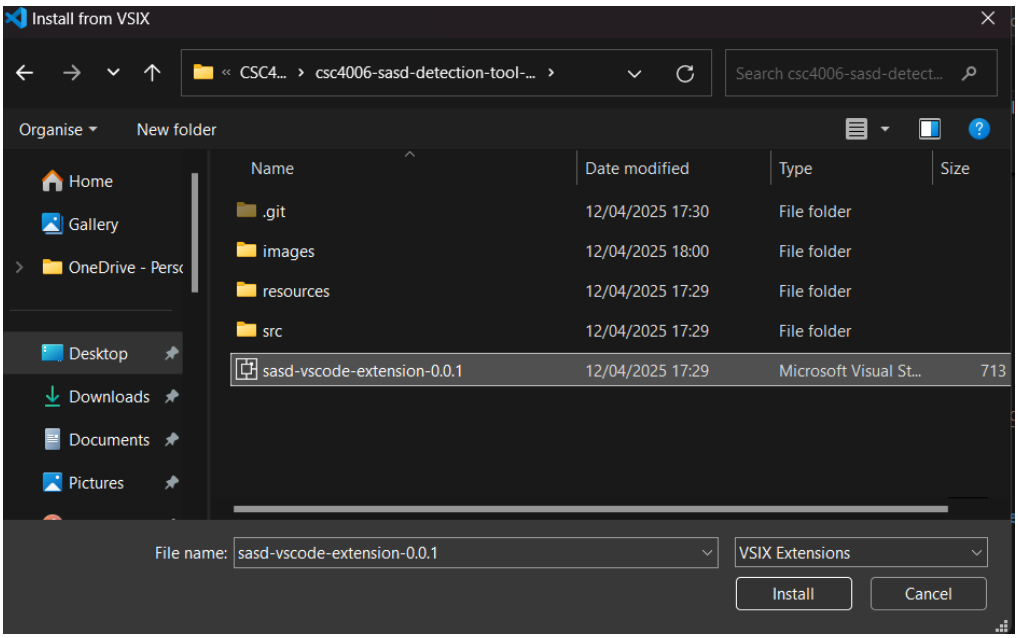
- Open Visual Studio Code and open Command Palette

```
Ctrl+Shift+P
```

- Search for Extensions: Install from VSIX



- Navigate to the Packaged VSCode Extension and Install



- Success: The VSCode Extension has been successfully installed once you see the SASD Detection Tool icon on the sidebar.

Usage

- ### Example Usage using React Frontend

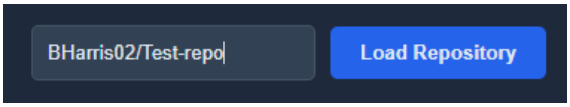
Below is an example usage of how to execute one function of the SASD Detection Tool using the React frontend. The execution of other functions of the tool follows this same process:

- **Navigate to:**

```
http://127.0.0.1:3000
```

This should display the frontend UI.

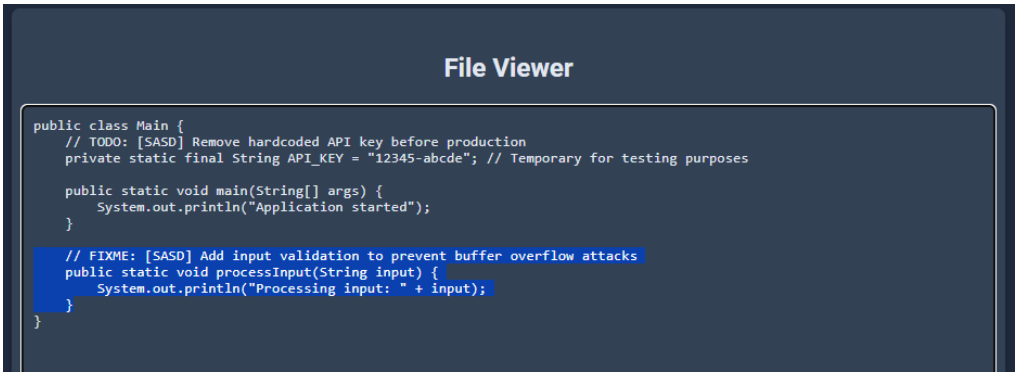
- **Search for a public repository:**



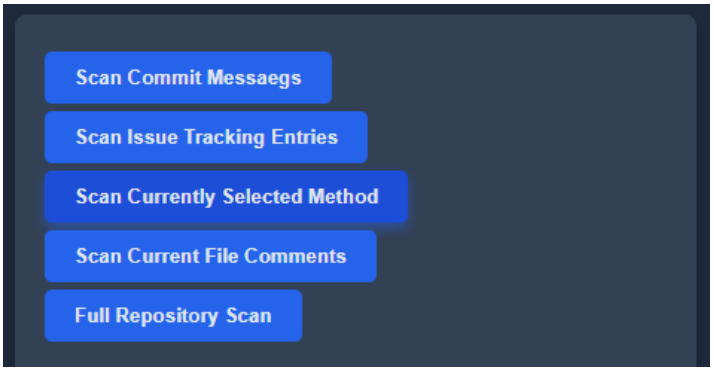
- **Select a file:**

This should display the file content.

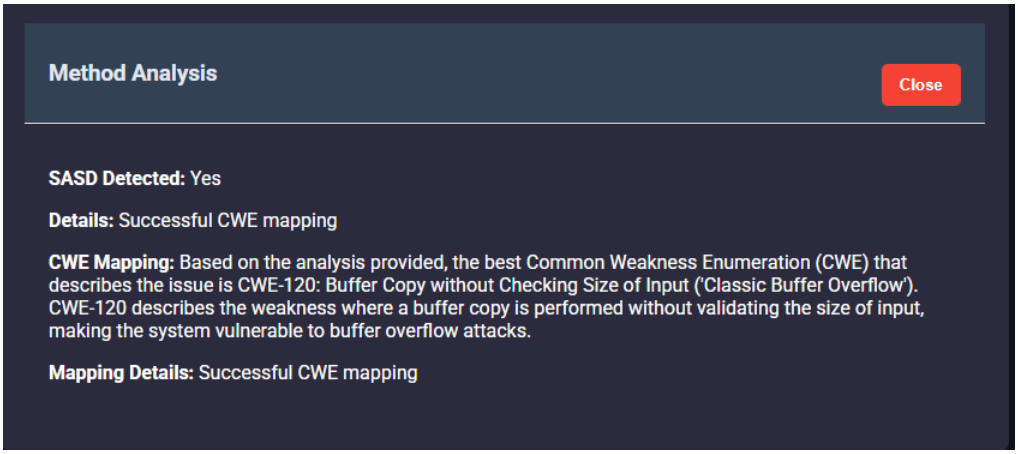
- **Highlight a Segment of Code:**



- **Execute the "Scan Currently Selected Method" function:**



- **View the Analysis Results:**



- Example Usage using VSCode Extension

Below is an example usage of one function of the SASD Detection Tool using the VSCode extension. The execution of other functions of the tool follow this same process:

- **Open a project using Visual Studio Code (This project must be a GitHub project with a remote origin):**
- **Open a file in the project with source code comments:**

app.rb

```ruby
# TODO: [SASD] Replace plaintext password with hashed storage
PASSWORD = "supersecret" # This is a security risk

# Function for processing user input
def process_user_input(input)
  # FIXME: Needs sanitisation to prevent XSS attacks
  puts "Processing input: #{input}"
end

# General greeting function
def greet
  puts "Hello, User!"
end
```

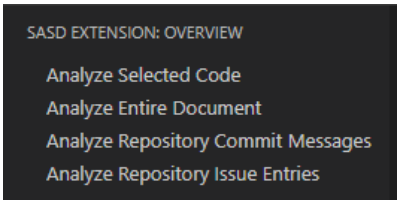
- **Open the Tool via the Icon:**

- **Highlight the desired segment of code to analyze:**

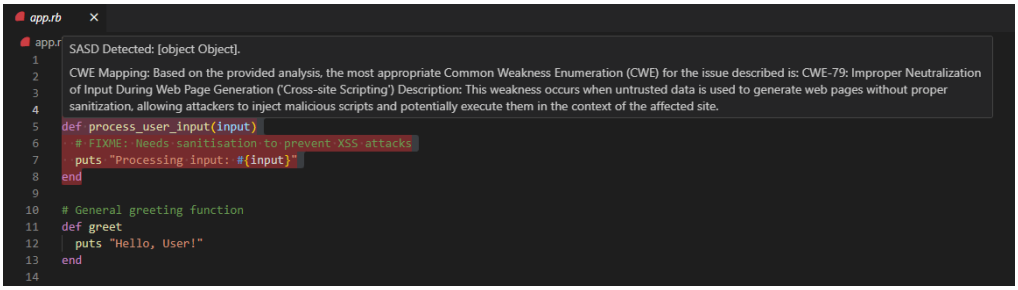
app.rb

```ruby
# TODO: [SASD] Replace plaintext password with hashed storage
PASSWORD = "supersecret" # This is a security risk

# Function for processing user input
def process_user_input(input)
  # FIXME: Needs sanitisation to prevent XSS attacks
  puts "Processing input: #{input}"
end

# General greeting function
def greet
  puts "Hello, User!"
end
```

- **Select the "Analyze Selected Code" Option in the Tool:**



- **View the Analysis Details:**



License

All components of the SASD Detection Tool are open source and released under the MIT License. See the License file for details.

Support

For support, questions, or to report issues, please contact the project maintainer directly:

- **Maintainer:** Blake Harris (bharris06@qub.ac.uk)

A.2 Other Component Repositories

In addition to the Orchestration Repository, each component of the SASD Detection Tool has its own dedicated repository. Their non-code documentation has already been covered in the Orchestration Repository README, however, should you wish to read these you can navigate to each individual repository:

- **Backend Repository**: `https://gitlab.eeecs.qub.ac.uk/40323251/CSC4006-SASD-Detection-Tool`
- **Frontend Repository**: `https://gitlab.eeecs.qub.ac.uk/40323251/csc4006-sasd-detection-tool-frontend`
- **VSCode Extension Repository**: `https://gitlab.eeecs.qub.ac.uk/40323251/csc4006-sasd-detection-tool-vscode-extension`

A.3 Results and Analysis Replication Guide

Additionally, there is a another repository containing the code used to conduct experiments and gather the results for the Result and Analysis section of the accompanying Research Article. Should you wish to replicate the results from the article, or run your own experiments, you can follow this guide:

README.md 2025-04-25

Detecting Self-Admitted Security Debt (SASD) in Software Projects Using Natural Language Processing (NLP) - Experiments

This repository contains the code used to conduct experiments and gather results detailed in the Results and Analysis section of the Research Article. This document will provide a guide to help you set up and replicate those results.

Contents

Prerequisites

- **Python 3.9 or higher**
- **pip (Python Package Installer)**
- **Git**
- **Backend Flask API**

Installation

In order to run the Experimental Pipeline, the backend Flask API will need to be set up and running.

Installing the Backend Flask API

If you have either the Flask API running as a standalone component, or have Dockerised the React frontend and Flask API into one unified deployment, you can skip this step.

1. **Clone the Repository:**

```
git clone https://gitlab.eeecs.qub.ac.uk/40323251/CSC4006-SASD-Detection-Tool.git
cd csc4006-sasd-detection-tool-backend
```

2. **Create a Virtual Environment:**

```
python -m venv venv
source venv/bin/activate    # Linux/MacOS
venv/scripts/activate       # Windows
```

3. **Install Dependencies:**

```
pip install -r requirements.txt
```

4. **Configuration**

Create a .env file in the project root directory. Use the following template:

```
FLASK_ENV=development

GITHUB_API_URL = "https://api.github.com"
GITHUB_API_TOKEN = <your_github_api_token>

GITHUB_CLIENT_ID = <your_github_client_id>
GITHUB_CLIENT_SECRET = <your_github_client_secret>

GITHUB_ACCESS_TOKEN_URL = "https://github.com/login/oauth/access_token"
GITHUB_API_USER_URL = "https://api.github.com/user"

OPENAI_MODEL = "gpt-3.5-turbo"
OPENAI_KEY = <your_openai_key>
```

- **GITHUB_API_TOKEN:** A personal access token from GitHub for API calls.
- **GITHUB_CLIENT_ID:** Unique ID for the signed-in user.
- **GITHUB_CLIENT_SECRET:** Used to get an access token for the signed-in user.
- **OPENAI_KEY:** Your OpenAI API key to perform tasks using NLP.

5. **Start the Flask Server:**

```
python main.py
```

Your Flask API will be available at:

```
http://127.0.0.1:5000
```

Installing the Experimental Pipeline

Once you have the Flask API set up and running, you can proceed with installing the Experimental Pipeline.

1. **Clone the Repository:**

```
git clone https://gitlab.eeecs.qub.ac.uk/40323251/csc4006-sasd-detection-tool-experimental-utilities.git
cd csc4006-sasd-detection-tool-experimental-utilities
```

2. **Configuration:**

Please ensure the address and port of the **API_URL** in the *config.py* file is the same as your running Flask API:

```
API_URL = "http://127.0.0.1:5000/api/analyze/method"
```

Usage

Important Note
As outlined in the Limitations subsection of the Research Article, older Natural Language Processing models, such as GPT-3.5-Turbo, have a tendency to produce slightly different responses each time they are posed with the same query. Due to this, if you wish to see the results portrayed in the Research Article, you should **NOT** run the commands to execute the experiment, but rather only execute the **analyses** and **graph** commands. Running the experiment commands will override the data used in the Result and Analysis Section. If you wish to replicate the results from the Research Article follow the **Replication Guide** steps, otherwise proceed with the **Running the Experiments** steps.

Replication Guide

1. **Detection Experiment Analysis:**

In order to see the raw metrics of the Detection Experiment, run the below command:

```
python main.py detection_analysis
```

2. **Mapping Experiment Analysis:**

In order to see the raw metrics of the Mapping Experiment, run the below command:

```
python main.py mapping_analysis
```

3. **Creating Experiment Results Graphs:**

In order to generate graphs based on the results, run the below command:

```
python main.py graphs
```

Running the Experiments

Follow this series of steps in order to produce a new set of results:

1. **Dataset Creation:**

Ensure you have a folder named **data** in the root directory. Ensure the *technical_debt_dataset.csv* is inside this folder (You can ignore any other files).

Run the **files** command to create a stratified, augmented sample of the dataset:

```
python main.py files
```

This should create (or replace) **three** files in the **data** folder:

- converted_dataset.py
- stratified_sample.csv
- stratified_sample_augmented.csv

Detection Experiment

Ensure your Flask API is running.

1. **Run the Detection Experiment:**

```
python main.py detection
```

This will take some time to run depending on your hardware.

Once complete, you will need to do manual verification on the **10** sets of verification samples (30 samples total).

Each Verification Sample contains the following:

- **comment:** The source code comment, issue or commit message.
- **baseline_result:** Whether or not the keyword-based baseline detection method detected SASD (True or False).
- **nlp_result:** Whether or not the NLP detection method detected SASD (True or False).
- **sasd:** This is where you determine if the **comment** indicates SASD (True). By default this will be False for all comments.

2. **Detection Experiment Analysis:**

Once you have manually verified all 30 samples, to see the raw metrics of the Detection Experiment, run the below command:

```
python main.py detection_analysis
```

Mapping Experiment

Ensure you Flask API is running.

In order to run the Mapping Experiment, please ensure you have **completed the Detection Experiment first**.

1. **Run the Mapping Experiment:**

```
python main.py mapping
```

This will create Verification samples.

2. **Run the Verification Command:**

In the *config.py* file you can find the *SYNTHETIC_SASD_COMMENTS_MAPPINGS* dictionary. If you wish to change the CWE mappings for each synthetic SASD comment in the dataset you can navigate here to do so before executing the Verification command.

```
python main.py mapping_verification
```

3. **Mapping Experiment Analysis:**

```
python main.py mapping_analysis
```

Generating Graphs

After running both experiments, you can execute the following command to create graphs:

```
python main.py graphs
```

Additional Information

For the experiments conducted to collect results for the Research Article the following seeds were used:

```
Detection Experiment Seeds:

Run 1: [42, 99, 123]
Run 2: [42, 99, 123]
Run 3: [271828, 314159, 161803]

Run 4: [42, 99, 123]
Run 5: [42, 99, 123]
Run 6: [123456, 789012, 345678]

Run 7: [42, 99, 123]
Run 8: [42, 99, 123]
Run 9: [202305, 987654, 135790]

Run 10: [42, 99, 123]
```

License

This software is licensed under the MIT License. See the LICENSE file for details.

Support

For support, questions, or to report issues, please contact the project maintainer directly:

- **Maintainer:** Blake Harris (bharris06@qub.ac.uk)

B Meeting Minutes

Below are minutes taken from meetings with the project supervisor from throughout the academic year:

QUEEN'S UNIVERSITY OF BELFAST
COMPUTER SCIENCE

Minute of Project Supervision Meeting

Student Name:	**Blake Harris**		
Project Module Code:	**CSC4006**		
Project Supervisor:	**Dr Desmond Greer**		
Meeting Number:	**1**	**Date of Meeting:**	**08-10-24**

Progress since last meeting, and decisions arrived at during meeting:

Initial meeting – some brief reading on technical debt (primarily research paper abstract and web resources)

OpenAI API is feasible for an NLP implementation
Brand new idea – not based on prior work
Word comparison w/ CWEs?

Action Points:

Further in-depth reading on technical debt
Formulate a Research question as a basis

Date of next meeting:

25-10-24

Agreed minute should be signed by the student and initialled by the supervisor.

Student's Signature: _____BHarris_____ **Date:** _____08-10-24__

Supervisor's Initials: _____ **Date:** _____

Supervisor's Comments:

QUEEN'S UNIVERSITY OF BELFAST
COMPUTER SCIENCE

Minute of Project Supervision Meeting

Student Name:	**Blake Harris**		
Project Module Code:	**CSC4006**		
Project Supervisor:	**Dr Desmond Greer**		
Meeting Number:	**2**	**Date of Meeting:**	**25-10-24**

Progress since last meeting, and decisions arrived at during meeting:

Deeper, more coherent understanding of SATD
Research questions formulated – evolution from basic to deep, complex and inquisitive
Software solution formulated

Action Points:

Hypothesis formulation – Null hypothesis specifically
Determine metrics to be measured/collected/analyzed
Search for datasets with SASD
Review RQs for simplicity

Date of next meeting:

Agreed minute should be signed by the student and initialled by the supervisor.

Student's Signature: _____BHarris_____ **Date:** _____25-10-24___

Supervisor's Initials: _____ **Date:** _____

Supervisor's Comments:

QUEEN'S UNIVERSITY OF BELFAST
COMPUTER SCIENCE

Minute of Project Supervision Meeting

Student Name:	**Blake Harris**		
Project Module Code:	**CSC4006**		
Project Supervisor:	**Dr Desmond Greer**		
Meeting Number:	**3**	**Date of Meeting:**	**01-11-24**

Progress since last meeting, and decisions arrived at during meeting:

Code created to retrieve git commit messages and issues, and source code comments + preprocessing data ready for NLP

Action Points:

Select and implement Hugging Face NLP models for SASD Detection and CWE Mapping.

Date of next meeting:

Agreed minute should be signed by the student and initialled by the supervisor.

Student's Signature: _____BHarris_____ **Date:** _____01-11-24____

Supervisor's Initials: _____ **Date:** _____

Supervisor's Comments:

QUEEN'S UNIVERSITY OF BELFAST
COMPUTER SCIENCE

Minute of Project Supervision Meeting

Student Name:	**Blake Harris**		
Project Module Code:	**CSC4006**		
Project Supervisor:	**Dr Desmond Greer**		
Meeting Number:	**4**	**Date of Meeting:**	**13-11-24**

Progress since last meeting, and decisions arrived at during meeting:

Preliminary research article first draft complete

Action Points:

Read through morse grant proposal sent by Des for additional points on Security debt. Investigate internal workings of existing tools for better understanding + use for better interpretation of final software solution.

Date of next meeting:

Agreed minute should be signed by the student and initialled by the supervisor.

Student's Signature: _____BHarris_____ **Date:** _____13-11-24___

Supervisor's Initials: _____ **Date:** _____

Supervisor's Comments:

QUEEN'S UNIVERSITY OF BELFAST
COMPUTER SCIENCE

Minute of Project Supervision Meeting

Student Name:	**Blake Harris**		
Project Module Code:	**CSC4006**		
Project Supervisor:	**Dr Desmond Greer**		
Meeting Number:	**5**	**Date of Meeting:**	**27-01-25**

Progress since last meeting, and decisions arrived at during meeting:

First iteration of software tool developed
Methodology section of report planned

Action Points:

Evaluation of tool > compare responses from different models
- > vulnerability tool

Experiments and results > original RQ too complex; decompose + need hypotheses

Other potential RQs: is SASD more likely in older projects or projects with more contributors?

Date of next meeting:

Agreed minute should be signed by the student and initialled by the supervisor.

Student's Signature: _____BHarris_____ **Date:** _____27-01-25____

Supervisor's Initials: _____ **Date:** _____

Supervisor's Comments:

QUEEN'S UNIVERSITY OF BELFAST
COMPUTER SCIENCE

Minute of Project Supervision Meeting

Student Name:	**Blake Harris**		
Project Module Code:	**CSC4006**		
Project Supervisor:	**Dr Desmond Greer**		
Meeting Number:	**6**	**Date of Meeting:**	**03-03-25**

Progress since last meeting, and decisions arrived at during meeting:

Research article: introduction and literature review rewritten + methodology section organized and drafted + experiments and metrics planned and required additional utility software developed.
Software development report first draft completed
Plans for v1.0.1 of SASD Detection tool planned

Action Points:

Conduct all experiments and gather results + Manually verify a subset of the stratified sample of the overall dataset to establish a (proxy) ground truth

Date of next meeting:

07-03-25

Agreed minute should be signed by the student and initialled by the supervisor.

Student's Signature: _____BHarris_____ **Date:** _____03-03-25_____

Supervisor's Initials: _____ **Date:** _____

Supervisor's Comments:

QUEEN'S UNIVERSITY OF BELFAST
COMPUTER SCIENCE

Minute of Project Supervision Meeting

Student Name:	**Blake Harris**		
Project Module Code:	**CSC4006**		
Project Supervisor:	**Dr Desmond Greer**		
Meeting Number:	**7**	**Date of Meeting:**	**11-03-25**

Progress since last meeting, and decisions arrived at during meeting:

Experiments: Code complete and results gathered and analyzed.
Brief reading on Synthetic Data Augmentation - implemented into experiment pipeline due to weak dataset.
Results gathered based on sub-samples of an overall stratified sample + extrapolated and scaled to full dataset.

Action Points:

Graphs, tables, etc for gathered results from experiments.

Write results and analysis section of Research Article.

Next iteration of software (bug fixes, actionable insights, Git OAuth for private repository analysis)

Date of next meeting:

Agreed minute should be signed by the student and initialled by the supervisor.

Student's Signature: _____BHarris_____ **Date:** _____11-03-25_____

Supervisor's Initials: _____ **Date:** _____

Supervisor's Comments:

QUEEN'S UNIVERSITY OF BELFAST
COMPUTER SCIENCE

Minute of Project Supervision Meeting

Student Name:	**Blake Harris**		
Project Module Code:	**CSC4006**		
Project Supervisor:	**Dr Desmond Greer**		
Meeting Number:	**8**	**Date of Meeting:**	**15-04-25**

Progress since last meeting, and decisions arrived at during meeting:

Research Article Completed
Software Development Completed
GitLab Documentation mostly completed
Software completed

Action Points:

Finish GitLab documentation - Frontend (all), REPLICATION (all)

Include Meeting Minutes in Software Development Report Appendices

Include GitLab Documentation in Software Development Report Appendices??

Software development report - fix images

Date of next meeting:

Agreed minute should be signed by the student and initialled by the supervisor.

Student's Signature: _____BHarris_____ **Date:** _____15-04-25_____

Supervisor's Initials: _____ **Date:** _____

Supervisor's Comments:

References

[1] Pydantic, "Pydantic documentation." `https://docs.pydantic.dev/latest/`, 2023. Python's Pydantic library documentation.